

Color code: major topics, important stuff

by Erik Girdwaskas

rolling cheat sheet - DDCA

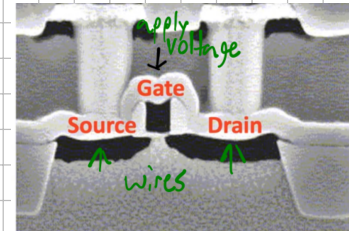
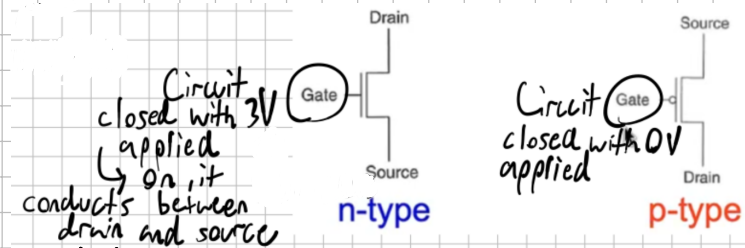
foreword: if there is one subject to trust me on, it's this! the sheet is based on my reading notes + the lectures

Basics/Boolean Algebra:

Transistors: The most basic building blocks of computers, they are mostly (only?) implemented with

MOS $\rightarrow$ metal, oxide (as insulator), semiconductors

$\rightarrow$ They are used to build logic gates, don't worry about constructing anything with them for the exam though!



$\rightarrow$ 3V is a stand-in for any HIGH voltage here

$\rightarrow$ think of n-type transistors as active high and active low components

$\rightarrow$ exercise: the 7-segment display in our FPGA displays a value when its transistors are driven to a LOW voltage. What transistors does it use?

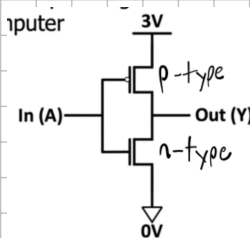
Logic Gates:

$\rightarrow$ They are basic modules made from transistors and boolean algebra is performed on them

$\rightarrow$ the term CMOS stands for complementary MOS, describing the use of both p-type and n-type transistors in larger structures like gates

NOT gate:

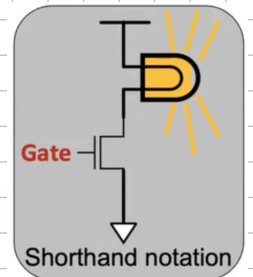
(logical inverted)



Truth Table (these all use truth tables):

A	P	N	Y
0	ON	OFF	1
1	OFF	ON	0

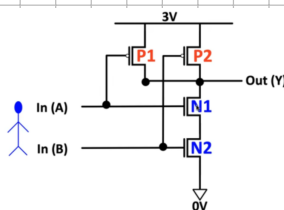
general structure $\rightarrow$ of gate schematic



$\leftarrow$ example path of 0V being passed as input

NAND gate:

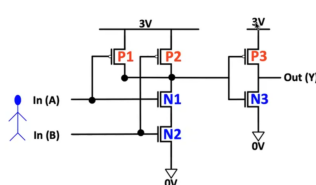
(not and)



TT:

A	B	P1	P2	N1	N2	Y
0	0	ON	ON	OFF	OFF	1
0	1	ON	OFF	OFF	ON	1
1	0	OFF	ON	ON	OFF	1
1	1	OFF	OFF	ON	ON	0

AND gate:



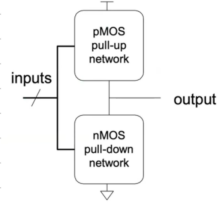
TT:

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

$\rightarrow$ Comment: see how we built this from a NAND gate and an inverter? This saves us transistors compared to building it from scratch

→ NAND and NOR are logically complete because you can build any Boolean expression using those two gates

CMOS gate structure:



→ Example: in XOR, only one network should be on, if both are on there's a short-circuit
if neither are, an undefined floating output is sent out

→ pMOS only carries 1's (HIGH voltage) well, nMOS only carries 0's (LOW voltage) well

→ this means that with pMOS, passing a logical LOW causes voltage loss (other way around for nMOS)

→ it's also the reason we need both (CMOS)

Electrical stuff (maybe exam relevant?)

→ you can compare running something in series electrically to a logical AND and electrical parallel to a logical OR

→ Latency: Series puts more resistance on the wire carrying it, and as resistance determines latency, series is slower

→ This is why practically, as little series as possible is used

→ Dynamic Power Consumption: power to charge a transistor while the signal changes (inversion): $C \cdot V^2 \cdot f$
Capacitance of capacitor, Voltage, Charge frequency

→ Static Power Consumption: power used while signals do not change: $V \cdot I_{\text{leakage}}$ ← current that keeps flowing, hard to minimize

→ Energy Consumption: Power · Time

→ at no point is the power used meant to exceed the Thermal Design Product (TDP, max. intended power)

Diagram for Gates and their Truth Tables (from now on, we won't use the transistor-level schematic)

Buffer

A	Z
0	0
1	1

AND

A	B	Z
0	0	0
0	1	0
1	0	0
1	1	1

OR

A	B	Z
0	0	0
0	1	1
1	0	1
1	1	1

XOR

A	B	Z
0	0	0
0	1	1
1	0	1
1	1	0

Inverter

A	Z
0	1
1	0

NAND

A	B	Z
0	0	1
0	1	1
1	0	1
1	1	0

NOR

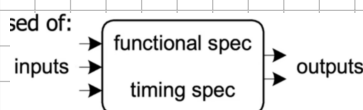
A	B	Z
0	0	1
0	1	0
1	0	0
1	1	0

XNOR

A	B	Z
0	0	1
0	1	0
1	0	0
1	1	1

Circuits:

Structure:



Functional spec.: putting outputs in terms of inputs (using the math. def. of function)

→ this uses Boolean algebra, which is mostly repetition from DiskMath

Axioms and laws: (+ replaces v from DM, · replaces ∧, $\bar{A}$ replaces $\neg A$)

All axioms are given in dual form:

- A dual of a Boolean expression is derived by replacing
 - Every AND operation with... an OR operation
 - Every OR operation with... an AND operation
 - Every constant 1 with... a constant 0
 - Every constant 0 with... a constant 1
 - But don't change any of the literals or play with the complements!

Example

$$a \cdot (b + c) = (a \cdot b) + (a \cdot c)$$

$$\rightarrow a + (b \cdot c) = (a + b) \cdot (a + c)$$

(Refers to the entire Boolean expression)

Formal version

1. B contains at least two elements, 0 and 1, such that $0 \neq 1$
2. Closure $a, b \in B$,
(i) $a + b \in B$
(ii) $a \cdot b \in B$
3. Commutative Laws: $a, b \in B$,
(i) $a + b = b + a$
(ii) $a \cdot b = b \cdot a$
4. Identities: $0, 1 \in B$
(i) $a + 0 = a$
(ii) $a \cdot 1 = a$
5. Distributive Laws:
(i) $a + (b \cdot c) = (a + b) \cdot (a + c)$
(ii) $a \cdot (b + c) = a \cdot b + a \cdot c$
6. Complement:
(i) $a + \bar{a} = 1$
(ii) $a \cdot \bar{a} = 0$

Operations with 0 and 1:

1. $X + 0 = X$
2. $X + 1 = 1$

Dual
1D. $X \cdot 1 = X$
2D. $X \cdot 0 = 0$

AND, OR with identities gives you back the original variable or the identity

Idempotent Law:

3. $X + X = X$

- 3D. $X \cdot X = X$

AND, OR with self = self

Involution Law:

4. $(\bar{X}) = X$

double complement = no complement

Laws of Complementarity:

5. $X + \bar{X} = 1$

- 5D. $X \cdot \bar{X} = 0$

AND, OR with complement gives you an identity

Commutative Law:

6. $X + Y = Y + X$

- 6D. $X \cdot Y = Y \cdot X$

Just an axiom...

Associative Laws:

7. $(X + Y) + Z = X + (Y + Z)$
 $= X + Y + Z$

- 7D. $(X \cdot Y) \cdot Z = X \cdot (Y \cdot Z)$
 $= X \cdot Y \cdot Z$

Parenthesis order does not matter

Distributive Laws:

8. $X \cdot (Y + Z) = (X \cdot Y) + (X \cdot Z)$

- 8D. $X + (Y \cdot Z) = (X + Y) \cdot (X + Z)$

Axiom

Simplification Theorems:

9. $X \cdot Y + X \cdot \bar{Y} = X$

- 9D. $(X + Y) \cdot (X + \bar{Y}) = X$

10. $X + X \cdot Y = X$

- 10D. $X \cdot (X + Y) = X$

11. $(X + \bar{Y}) \cdot Y = X \cdot Y$

- 11D. $(X \cdot \bar{Y}) + Y = X + Y$

Useful for simplifying expressions

DeMorgan's Law:

$$12. \overline{(X + Y + Z + \dots)} = \bar{X} \cdot \bar{Y} \cdot \bar{Z} \cdot \dots$$

$$12D. \overline{(X \cdot Y \cdot Z \cdot \dots)} = \bar{X} + \bar{Y} + \bar{Z} + \dots$$

Canonical function forms:

Sum of Products (SOP, identical to DNF from DiskMath):

→ Two-level logic: at most two levels of gates → no path goes through more than 2 gates (SOP or Product of Sums (POS))

→ all Boolean equations can be put into SOP and POS

→ deriving it from a truth table: find every entry that evaluates to 1, make these an implicant (connected with $\cdot$), then make a disjunction of implicants (connect with $+$)

→ each implicant here is a minterm (product of all input variables)

→ standard notation: enumerate over minterms based on the binary number created by the input pattern

→ Example: $\sum m(0, 6, 7)$

POS: (identical to CNF)

→ deriving it from a truth table: take each expression that evaluates to false, disjoin them (connect with $+$, this makes them maxterms (sum of all input variables) and conjoin the maxterms (using $\cdot$)

→ standard notation: similar to SOP, but take the product of maxterms (lire: $\prod M(0, 6, 7)$)

Conversion rules:

1. **Minterm to Maxterm conversion:**
rewrite minterm shorthand using maxterm shorthand
replace minterm indices with the indices not already used
E.g., $F(A, B, C) = \sum m(3, 4, 5, 6, 7) = \prod M(0, 1, 2)$
2. **Maxterm to Minterm conversion:**
rewrite maxterm shorthand using minterm shorthand
replace maxterm indices with the indices not already used
E.g., $F(A, B, C) = \prod M(0, 1, 2) = \sum m(3, 4, 5, 6, 7)$

3. **Expansion of F to expansion of $\bar{F}$:**

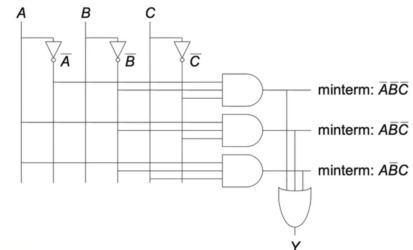
$$\begin{aligned} \text{E.g., } F(A, B, C) &= \sum m(3, 4, 5, 6, 7) \longrightarrow \bar{F}(A, B, C) = \sum m(0, 1, 2) \\ &= \prod M(0, 1, 2) \longrightarrow = \prod M(3, 4, 5, 6, 7) \end{aligned}$$

4. **Minterm expansion of F to Maxterm expansion of $\bar{F}$:**
rewrite in Maxterm form, using the same indices as F

$$\begin{aligned} \text{E.g., } F(A, B, C) &= \sum m(3, 4, 5, 6, 7) \longrightarrow \bar{F}(A, B, C) = \prod M(3, 4, 5, 6, 7) \\ &= \prod M(0, 1, 2) \end{aligned}$$

■ SOP (sum-of-products) leads to two-level logic

■ Example: $Y = (\bar{A} \cdot \bar{B} \cdot \bar{C}) + (A \cdot \bar{B} \cdot \bar{C}) + (A \cdot \bar{B} \cdot C)$



Combinational Blocks:

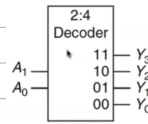
→ constructed using gates, they are a layer of abstraction above gates, so gate-level details are hidden

→ Note: Combinational Logic has no memory

Decoder: Has a separate output for every possible input pattern, n inputs, 2^n outputs

2-to-4 decoder:

A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0



In Verilog:

Verilog

```
module decoder #(parameter N = 3)
  (input [N-1:0] a,
   output reg [2**N-1:0] y);

  always @ (*)
  begin
    y = 0;
    y[a] = 1;
  end
endmodule
```

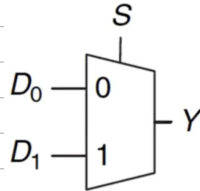
set output indexed at numerical value of input a to 1

→ oh you will see how it comes up in a CPU

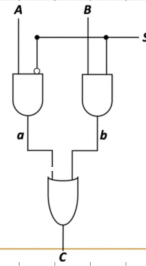
Multiplexer (MUX): Core idea: select one of N inputs to mirror to the output, decision is made by the $\log_2 N$ -bit control input:

2-to-1 MUX:

S	D_1	D_0	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1



(gate-level):



In Verilog (2-to-1 MUX):

Verilog

```
module mux2
  #(parameter width = 8)
  (input [width-1:0] d0, d1,
   input [width-1:0] s,
   output [width-1:0] y);

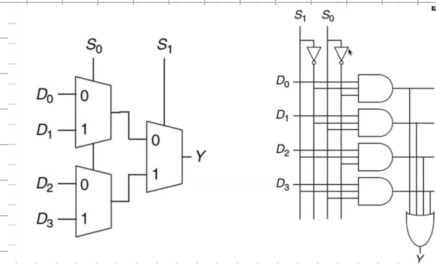
  assign y = s ? d1 : d0;
endmodule
```

default value 0
depends on width
we use assign w variable to decide module's width

Simpler TT:

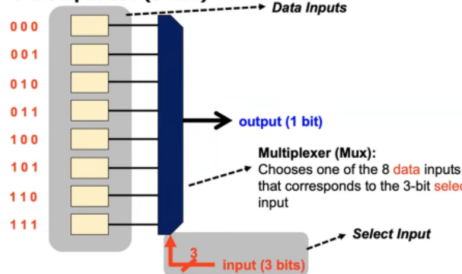
S	C
0	A
1	B

4-to-1 mux:



→ Lookup Tables (LUT), which our FPGAs use prominently for reprogrammable comb. logic, are implemented as trees of MUXes

3-bit input LUT (3-LUT)



3-LUT can implement any 3-bit input function

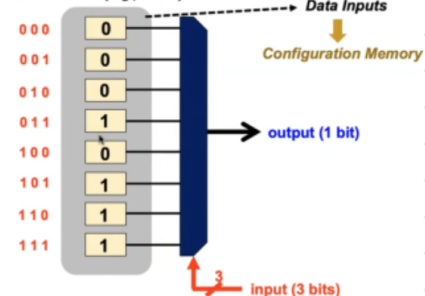
A function that outputs '1' when there are at least two '1's in a 3-bit input

In C:

```
int count = 0;
for(int i = 0; i < 3; i++) {
  count += input & 1;
  input = input >> 1;
}
if(count > 1) return 1;
return 0;

switch(input){
  case 0:
  case 1:
  case 2:
  case 4:
    return 0;
  default:
    return 1;
}
```

In Hardware (e.g., FPGA):



Full Adder: our way of performing bitwise addition on two inputs (A, B) with a carry C

gate-level:

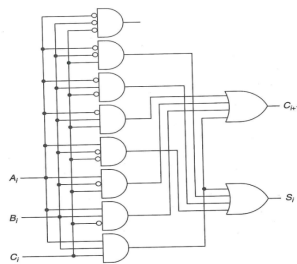
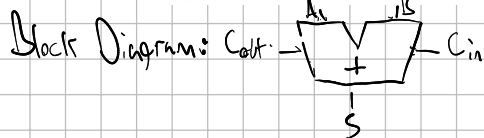


Figure 3.15 Gate-level description of a full adder



→ AND gates for each combination of "current-bit" inputs,

OR gates for "current-bit" output S_i and "next-bit" carry C_{i+1}

TT:

a_i	b_i	$carry_i$	$carry_{i+1}$	s_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

(Optional explanation)

For multi-bit addition, the most efficient CL (comb. logic) block is the Carry Lookahead Adder:

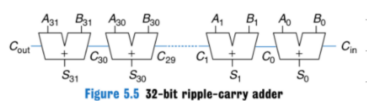


Figure 5.5 32-bit ripple-carry adder

→ in H&H Chapter 5, you see the Ripple-Carry Adder, which has more latency as the addition is done sequentially from the MSB (most significant bit) to the LSB (least significant bit), so the delay T_{ripple} grows with the number of bits (linearly)

The Carry Lookahead Adder (CLA) eliminates this delay:

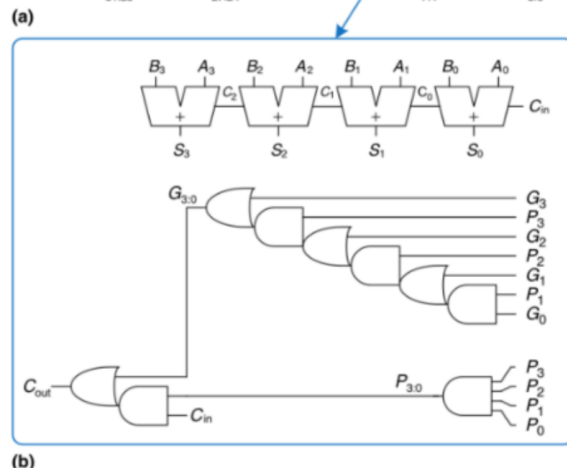
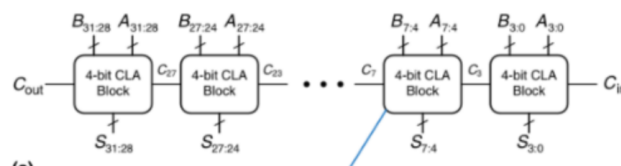
- the adder is divided into blocks
- circuitry can quickly determine C_{out} of a block with known C_{in}

→ Generate (G) and Propagate (P) signals describe how C_{out} is determined

→ the i -th column of the CLA generates a carry-out if it produces one independent of C_{in} → C_i is generated if $A_i = 1$ and $B_i = 1$ ⇒ $A_i \cdot B_i$

→ it propagates a carry-out if $A_i = 1$ or $B_i = 1$ ⇒ $A_i + B_i$

→ the i -th column generates C_i if it generates a carry or propagates a carry in ($P_i \cdot C_{i-1}$) ⇒ $C_i = A_i \cdot B_i + (A_i + B_i) \cdot C_{i-1}$
 $= G_i + P_i \cdot C_{i-1}$



→ our definitions from now extend to multi-bit blocks: a block generates a carry if it produces one independent of C_{in} and propagates one if a carry is produced under the condition of $C_{in} = 1$

→ G and P extend to $G_{i:j}$ and $P_{i:j}$ for the block $i:j$

→ pattern: Block generates a carry if most sig. column generates one or the most sig. column generates one and the next sig. column generates one...

$$G_{3:0} = G_3 + P_3(G_2 + P_2(G_1 + P_1 G_0))$$

→ it propagates one if all columns propagate: $P_{3:0} = P_3 P_2 P_1 P_0$

→ Carry formula: $C_{i:j} = G_{i:j} + P_{i:j} C_j$

→ single-bit and block generate/propagate is computed simultaneously

→ Critical path (defined later): Compute $G_{3:0}$ and G_0 in the first block, then advance sequentially

→ Timing definition (also formally introduced later): $t_{CLA} = t_{pg} + t_{pg_block} + (\frac{N}{K} - 1) \cdot t_{AND_OR} + K \cdot t_{FA} \leftarrow$ for full adders
 (for N bits in K-bit blocks)
 individual P, G gates (single AND/OR construct)

Verilog

```
module adder #(parameter N = 8)
    (input [N-1:0] a, b,
     input cin,
     output [N-1:0] s,
     output cout);

    assign {cout, s} = a + b + cin;
endmodule
```

↑ bundled → $\Sigma a, b, c$ parameters

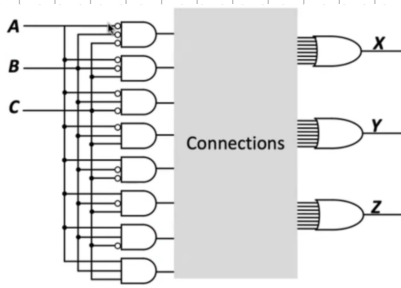
Verilog adder,

I will just continuously leave these on the notes to comb. blocks

→ we can determine dependencies (carries) based off just our block's information, allowing for parallelization
 → keep in mind that there are always tradeoffs: CLA is faster than ripple, but requires more complex hardware and area

Programmable Logic Array (PLA):

→ Universal structure allowing execution of any n -input logic function



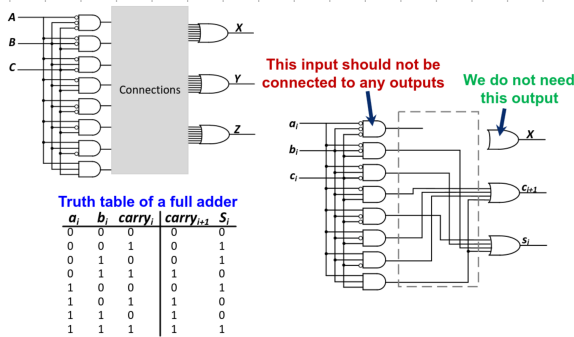
amount of AND gates → amount of possible SOP minterms → 2^n

OR gates → amount of output columns (we call it M) ($\leq 2^n$)

programming them: program connections from AND gates to OR gates

→ you can use this on truth tables

Example:

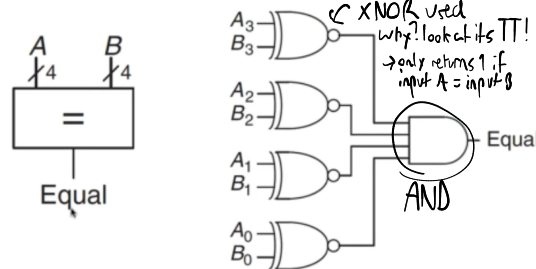


Logical Completeness: if a set of gates is logically complete, you can implement any logical function with it

→ {AND, OR, NOT} is log. complete → Example: PLA

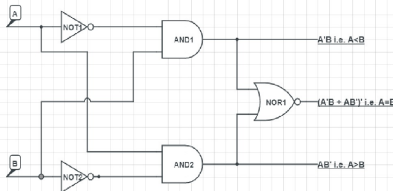
→ {NAND, NOR} are also log. complete

Comparator:



(Our first lab !!)

Magnitude comparator:



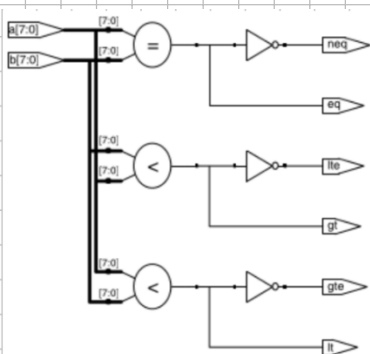
N-bit magnitude comparator in Verilog:

synthesizes to

this style is known as behavioral Verilog, we will go into it in a bit

```
module comparators # (parameter N = 8)
    (input [N-1:0] a, b,
     output
     eq, neq,
     lt, lte,
     gt, gte);
```

```
    assign eq = (a == b);
    assign neq = (a != b);
    assign lt = (a < b);
    assign lte = (a <= b);
    assign gt = (a > b);
    assign gte = (a >= b);
endmodule
```



ALU: (Arithmetic Logic Unit)

→ this is crucial to CPUs

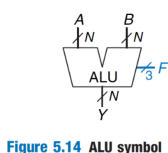


Figure 5.14 ALU symbol

Table 5.1 ALU operations

$F_{2:0}$	Function
000	A AND B
001	A OR B
010	A + B
011	not used
100	A AND $\bar{B}$
101	A OR $\bar{B}$
110	A - B
111	SLT

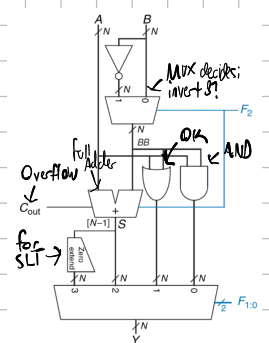
→ operations vary depending on your microarchitecture

→ our lab one had XOR and NOR

aside: 2's complement → negative binary numbers start with 1, zero is always a N-bit zero

→ this is how ALU's do math usually: convert a positive like 0101 to negative → invert bits, then add 1 → 1011 (-5)

Implementation:



Tri-State Buffer:

Tristate Buffer

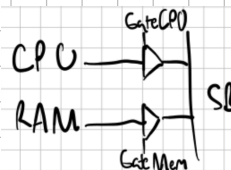


E	A	Y
0	0	0
0	1	1
1	0	0
1	1	1

Figure 2.40 Tristate buffer

→ z is a floating value that is neither 1 nor 0 (more later)

→ example usage: a shared bus → beautiful diagram:



Simplify TT:

E	Y
0	z
1	A

→ when CPU in use, we set GetMem to 0, it pushes z to the bus, which doesn't interfere with the CPU value

cool usage: MUX using tri-state buffers:

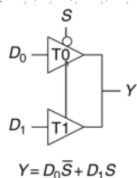
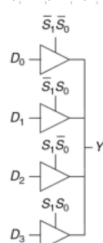


Figure 2.56 Multiplexer using tristate buffers



Logic Simplification:

→ why? we want to reduce the number of gates/inputs to save space/latency

→ Uniting Theorem: find "ON" set of function, if an input can change without changing the output, its value is not needed

Key Tool: The Uniting Theorem — $F = \overline{A}B + AB$

A	B	F
0	0	0
0	1	0
1	0	1
1	1	1

$$F = \overline{A}B + AB = A(\overline{B} + B) = A(1) = A$$

B's value changes within the rows where $F=1$ ("ON set")

A's value does NOT change within the ON-set rows

If an input (B) can change without changing the output, that input value is not needed

→ B is eliminated, A remains

→ B is eliminated, A remains

A	B	G
0	0	1
0	1	0
1	0	1
1	1	0

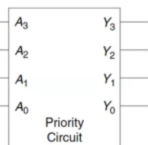
$$G = \overline{A}B + AB = (\overline{A} + A)B = B$$

B's value stays the same within the ON-set rows

A's value changes within the ON-set rows

→ A is eliminated, B remains

Priority Circuit: Inputs: requestors, Output: who gets priority:



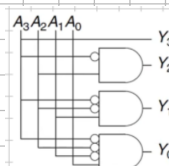
A ₃	A ₂	A ₁	A ₀	Y ₃	Y ₂	Y ₁	Y ₀
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	0
0	0	1	1	0	0	1	0
0	1	0	0	0	1	0	0
0	1	0	1	0	1	0	0
0	1	1	0	0	1	0	0
0	1	1	1	0	1	0	0
1	0	0	0	1	0	0	0
1	0	0	1	1	0	0	0
1	0	1	0	1	0	0	0
1	0	1	1	1	0	0	0
1	1	0	0	1	0	0	0
1	1	0	1	1	0	0	0
1	1	1	0	1	0	0	0
1	1	1	1	1	0	0	0

→ simplify:

A ₃	A ₂	A ₁	A ₀	Y ₃	Y ₂	Y ₁	Y ₀
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	X	0	0	1	0
0	1	X	X	0	1	0	0
1	X	X	X	1	0	0	0

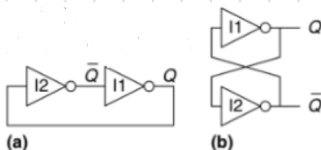
Figure 2.29 Priority circuit truth table with don't cares (X's)

circuit:



Sequential Logic:

Cross-Coupled Inverter:



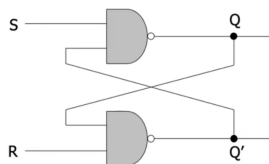
→ Q and $\overline{Q}$ depend on each other

Case 1: $Q=0$: I₂ receives false → $\overline{Q}$ turns true, I₁ receives false → Q false ...

Case 2 analogous → the circuit has 2 stable states (bistable)

→ Metastable state: both outputs halfway between 0 and 1 (explained later)

→ we have no control mechanism, so this concept is expanded into the R-S latch:



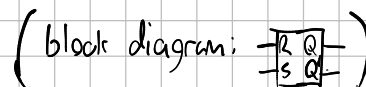
Input		Output
R	S	Q
1	1	Q_{prev}
1	0	1
0	1	0
0	0	Forbidden

→ instead of inverters, we cross-coupled NAND gates (book uses NOR)

→ Data stored at Q, its inverse at $\overline{Q}$

→ R and S are control inputs (quiescent/idle) state: both are 1

→ S(set): you set by driving S to 0 and not resetting (keep R at 1)

(block diagram: )

→ R(reset): you reset by driving R to 0 and not setting (keep S at 1)

→ S and R should not both be at 0, why?

→ logically: you cannot both set and reset at once, this also causes both Q and Q' to be 1, breaking the invariant (here: meaning of Q)

→ practically: if both Q and Q' are set to 1 at the same time, Q and Q' start to oscillate (metastability, values depend on each other)

→ they would just start wildly switching between 1 and 0 because Q cannot equal Q'

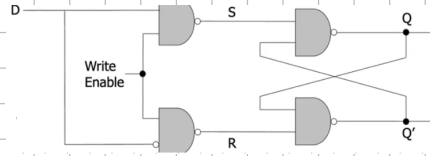
→ the time this takes to resolve varies (seen later)

→ The gated D latch guarantees correct operation:

→ book version uses a clock instead of WE

(block diagram:)

→ D is our only real input (latch), if WE is enabled, Q takes the value of D

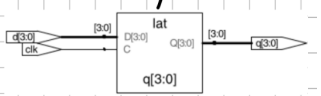


Input		Output
WE	D	Q
0	0	Q _{prev}
0	1	Q _{prev}
1	0	0
1	1	1

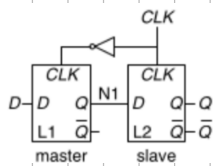
Verilog latch →

```
Verilog
module latch (input
               [3:0] d,
               input
               [3:0] q);
    always @ (clk, d)
        if (clk) q <= d;
endmodule
```

but in synthesis:



D flip-flop: (I'll use the clock version, you can imagine CLK as WE for now)



→ The reason we use this problematic terminology is for the following analogy: "slave copies master"

→ L1 and L2 are D latches, L1 is the master, L2 is the slave

→ CLK = 0 → L1 is transparent (taking in the value from D), L2 opaque (keeping its value)

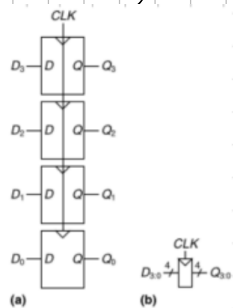
→ CLK = 1 → L1 is opaque, L2 is transparent (N1 is cut off from D and takes the value stored in Q)

→ in actions: L1 copies to L2 at the rising edge (CLK goes from 0 → 1) and remembers its state otherwise

Block diagram:



N-bit register: bank of N flip-flops



→ they share CLK and all flip-flops are updated at the rising edge

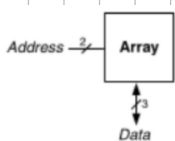
Adjacent concepts:

Enabled flip-flop → WE decides if Q is written on the rising edge

Resettable flip-flop → RESET input forces Q = 0

→ active low input → something happens after it is at 0

Memory arrays: shared properties of all memories:



→ memory is organized as a 2D array of memory cells

→ a program reads/writes the contents of an array row

→ rows are addressable → an array with N-bit addresses and M-bit data

has 2^N rows and M columns

→ each row is called a word → there are 2^N M-bit words

→ depth → #rows, width → #columns (word size), size → depth · width



→ memory arrays are built as arrays from bit cells

→ each bit cell stores 1 bit of data and is connected to both the wordline and bitline

→ for each combination of address bits, the memory asserts a single wordline that activates all bit cells in the row

→ wordline is HIGH → stored bit is transferred to or from the bitline

→ else, the bitline is not connected to the bit cell

→ usually, the bitline is at $\bar{Z}$ to read, when the wordline is ON, it is set to ON → this connects bitline to stored bit

→ this causes the bitline to overpower and rewrite the bit cell's data

Memory Arrays:

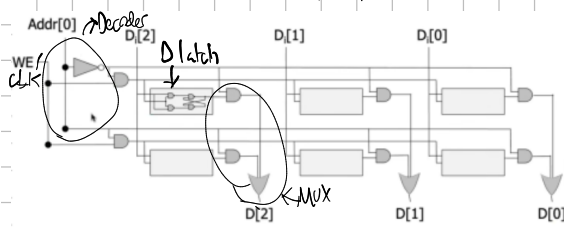
→ read: wordline asserted, bit cells then drive their lines H/L

→ write: drive bitlines H/L, then assert wordline to store the new values

→ ports: all memories have ≥ 1 port for read and write access each

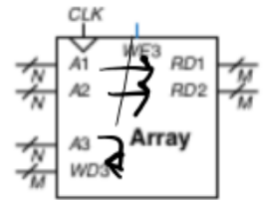
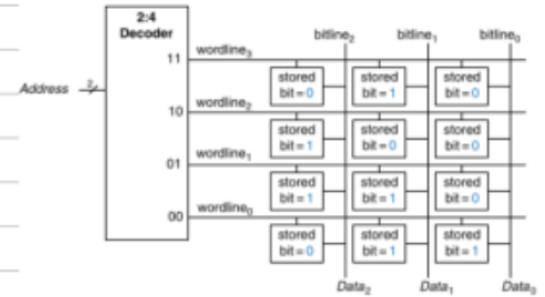
→ a multiported memory allows simultaneous reading/writing

The lectures used a slightly different design:



→ here, the point is to set individual cells rather than rows

→ decoder to decide row, MUX to decide current row → select input



Memory types: (NOTE: this is part 1, part 2 will come around W11-W12 → caches, memory banking, ...)

DRAM: a bit is stored as the presence or absence of charge in a capacitor

→ nMOS transistor behaves as switch between capacitor and bitline

→ wordline asserted → nMOS turns ON → bit value put on/taken from bitline

→ it is not driven high/low by a transistor tied to V_{DD} or GND (dynamic RAM)
voltage needed for HIGH ↑ ground, voltage for LOW

→ on read: data value driven from capacitor to bitline, write: opposite

→ charge leaks over time → DRAM must be reset every few ms when not read

SRAM: static, does not need to be refreshed

→ data bit stored in cross-coupled inverter with outputs bitline, $\overline{\text{bitline}}$ (as Q, $\overline{Q}$)

→ wordline asserted → both nMOS turn on, values are either set or gotten

→ if noise degrades charge, cross-coupled inverters restore values

→ latency and throughput depend on array size

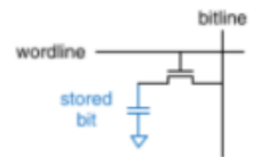


Figure 5.44 DRAM bit cell

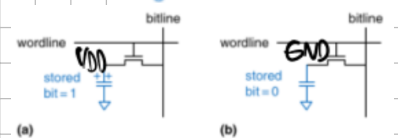


Figure 5.45 DRAM stored values



Figure 5.46 SRAM bit cell

Memory Type	Transistors per Bit Cell	Latency
flip-flop	~20	fast
SRAM	6	medium
DRAM	1	slow

important, DRAM is cheapest and slowest

Register files: systems hold temporary values there, it's a small, multiport SRAM array, cheaper than flip-flops

→ you will see that caches are basically fully SRAM

ROM: ~ bit is the presence or absence of a transistor

→ reading a cell: when the wordline of a row is ON, pull the bitline HIGH (wantly)

→ if a transistor exists, the corresponding bitline is driven LOW (absence means binary 0)

→ a ROM bit cell is a combinational circuit (no state to remember)

→ any ROM can be constructed from a decoder and NOT gates

→ ROM can perform any two-level logic function (→ recap: level:)

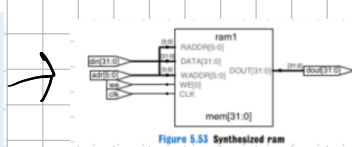
→ traditional ROM is hardcoded in manufacturing

→ if the transistors are always there, but disconnectible to ground, we have Programmable ROM (PROM)

→ if instead of transistors, we use floating gates, we have (Electrically) Erasable PROM (EEPROM/EPROM depending on technical details)

In Verilog:

```
module rom# (parameter N = 6, M = 32)
  (input clk,
   input we,
   input [N-1:0] adr,
   input [M-1:0] din,
   output [M-1:0] dout);
  reg [M-1:0] mem [2*N-1:0];
  always @ (posedge clk)
    if (we) mem[2*adr] <= din;
  assign dout = mem[2*adr];
endmodule
```



```
module rom (input [1:0] adr,
            output reg [2:0] dout);
  always @ (adr)
    case (adr)
      2'b00: dout <= 3'b011;
      2'b01: dout <= 3'b110;
      2'b10: dout <= 3'b100;
      2'b11: dout <= 3'b010;
    endcase
endmodule
```

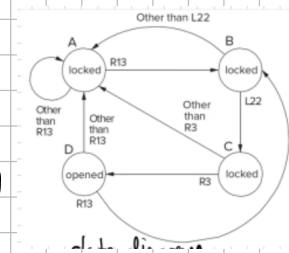
Back to sequential circuits:

State: this remembers past events and contributes to the output depending on current inputs

Analogy: Combinational lock:

State A: lock not open, 0/3 parts
 B: locked, 1/3 complete
 C: locked, 2/3 complete
 D: unlocked

→ state diagram for combination R13, L22, R3
 (it's one of the locks you turn left/right)



Asynchronous vs. synchronous:

→ async.: immediate state transition after input, state transitions are synchronized by nothing

→ sync.: state transitions at fixed times

→ done in most modern computers, it is much simpler to design, but input is delayed (usually by picoseconds today?)

→ in our course, we always use a clock to synchronize (you've seen this in the memory chapter already)

Clock: CLK: 1
 0 (0 → 1) (1 → 0)

→ state only changes at the rising and falling edges in
 ~ clock-controlled circuit

